



**PackSwift: A Hybrid Travel and
E-Commerce Platform with
Context-Aware Packing Recommendation
and Split-Payment Checkout**

Khaing Khant Kyaw

6605140002

**AN INDEPENDENT STUDY SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE
BACHELOR DEGREE OF INFORMATION TECHNOLOGY
SIAM UNIVERSITY**

2026



PackSwift: A Hybrid Travel and E-Commerce Platform with Context-Aware
Packing Recommendation and Split-Payment Checkout

KHAING KHANT KYAW

This Independent Study Has Been Approved as a Partial Fulfillment of the
Requirements for the Bachelor Degree of Information Technology

Advisor

Date...../...../.....

Siam University, Bangkok, Thailand

ABSTRACT

Title: PackSwift: A Hybrid Travel and E-Commerce Platform with Context-Aware Packing Recommendation and Split-Payment Checkout

By: Khaing Khant Kyaw

Degree: Bachelor Degree of Information Technology

Major: IT

Advisor: [Dr. Parham Porouhan \(Ph.D.\)](#)

This study presents the design and methodology of PackSwift, a hybrid travel and e-commerce web platform developed to address the fragmented nature of trip preparation. In conventional travel planning, users often move across multiple digital platforms to book flights, reserve hotels, check destination weather, and purchase essential travel items. This fragmented workflow increases time consumption, cognitive burden, and the risk of forgetting important items required for a specific trip. To address this problem, the proposed system integrates travel booking with a contextual product recommendation engine and a unified checkout interface.

The platform is designed to allow users to input destination, travel dates, trip purpose, and traveler composition, including adults, children, and pets. After flights and hotels are selected, the system generates highly targeted packing and shopping suggestions using destination data, weather conditions, travel timing, and traveler characteristics. The recommendation engine follows a rule-based methodology that maps trip context to product categories and filters items further by stock status and shipping feasibility. To maintain technical realism, the checkout process is designed as a single interface supported by split-payment routing, in which travel-related charges are routed through provider booking flows while retail-item charges are processed through the platform backend.

This report adopts a backend-first development methodology using Node.js, Express, Vanilla JavaScript, HTML, CSS, and MySQL. The study covers the system architecture, database schema, recommendation logic, split-payment structure, and phased implementation plan. The proposed platform contributes a practical model for combining travel services and contextual e-commerce within one minimalist digital workflow.

Keywords: hybrid travel platform, e-commerce, travel recommendation, packing assistant, split-payment routing, Node.js, MySQL, contextual recommendation



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my advisor, [Dr. Parham Porouhan \(Ph.D.\)](#), for the valuable guidance, constructive suggestions, and continuous support provided throughout the preparation of this report. The advisor's encouragement and academic direction greatly contributed to the completion and improvement of this study.

I would also like to extend my appreciation to the lecturers and staff of Siam University for their support throughout my studies. Their teaching, professional insight, and academic commitment have played an important role in my learning and development.

Finally, I am deeply grateful to my family, friends, and colleagues for their patience, encouragement, and moral support during the preparation of this project. Their confidence in me has been a constant source of motivation.

Khaing Khant Kyaw

DECLARATION

I, Khaing Khant Kyaw, hereby declare that the work embodied in this report entitled “PackSwift: A Hybrid Travel and E-Commerce Platform with Context-Aware Packing Recommendation and Split-Payment Checkout” is the result of my own original study and has not been submitted, either in part or in full, for any other degree or academic qualification at any other university or institution.

Khaing Khant Kyaw

28th April 2026



Table of Contents

ABSTRACT	3
ACKNOWLEDGEMENT	5
DECLARATION	6
CHAPTER ONE INTRODUCTION	8
1.1 Background of the Study	8
1.2 Problem Statement	8
1.3 Aim of the Project	9
1.4 Objectives of the Study	9
1.5 Motivation for the Study	10
1.6 Scope of the Study	11
1.7 Beneficiaries of the Study	12
1.8 Distinctiveness of the Project	12
1.9 Limitations of the Study	13
1.10 Summary of Chapter One	14
CHAPTER TWO RELATED WORKS / LITERATURE REVIEW	15
2.1 Introduction	15
2.2 Foundational Studies in Travel Recommendation	15
2.3 Personalized Travel Package Recommendation	17
2.4 Context-Aware Tourism Recommendation	17
2.5 Location-Based and Group Travel Recommendation	18
2.6 Packing and Baggage Recommendation Research	18
2.7 Related Commercial Systems	19
2.8 Research Gap and Positioning of the Proposed System	19
2.9 Summary of Chapter Two	20
REFERENCES (for Chapter Two)	21
CHAPTER THREE METHODOLOGY	23
3.1 Introduction	23
3.2 Development Approach	23
3.3 System Architecture and Design	24
3.3.1 Architectural Layers	24
3.3.2 End-to-End Data Flow	25

3.4 Technology Stack and Tools	26
3.4.1 Node.js for Backend Execution	26
3.4.2 Express for Routing and Middleware	26
3.4.3 Vanilla JavaScript, HTML, and CSS for Frontend Delivery	27
3.4.4 MySQL for Relational Data Storage	27
3.5 Algorithms and Logic of the Recommendation Engine	27
3.5.1 Purpose of the Recommendation Engine	27
3.5.2 Input Variables	28
3.5.3 Product Metadata Required	29
3.5.4 Rule Construction	30
3.5.5 Product Matching Logic	32
3.5.6 Scoring Formula	33
3.5.7 Example	34
3.6 Database Schema and Storage Design	35
3.6.1 Core Tables	35
3.6.2 Relationship Structure	36
3.6.3 Why Relational Storage Is Appropriate	38
3.7 Datasets and External Resources	38
3.7.1 Travel Data	38
3.7.2 Weather and Seasonal Data	39
3.7.3 Product Catalog Data	39
3.7.4 Loyalty Data	41
3.8 Development Planning	42
3.8.1 Phase One: Backend Foundation — Database Schema and Node.js Server Setup	42
3.8.2 Phase Two: Backend Logic — Travel APIs and Recommendation Engine	44
3.8.3 Phase Three: Backend Transactions — Cart, Pricing, and Split-Payment Routing	47
3.8.4 Phase Four: Frontend Interface — Trip Input and Travel Selection Screen	50
3.8.5 Phase Five: Frontend Interface — Smart Packing Recommendations Screen	52
3.8.6 Phase Six: Frontend Interface — Unified Checkout Screen	55
3.9 Summary of Chapter Three	57
REFERENCES	58

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Contemporary trip preparation is often fragmented across multiple digital platforms. A traveler may search for flights on one website, reserve accommodation on another, check destination weather on a separate service, and then visit several retail stores to purchase trip-related items. This process is inefficient because it separates booking decisions from practical preparation. At the same time, the travel sector remains strong and travelers continue to seek convenience, personalization, and efficient digital journeys. McKinsey's 2024 tourism analysis reports continued growth in travel demand and highlights the importance of evolving traveler expectations and digital experiences.

The proposed project addresses this gap through a hybrid travel and e-commerce web application. The system allows users to book flights and hotels, then immediately receive shopping recommendations tailored to the trip's destination, expected weather, and traveler composition. This approach converts trip booking from a standalone transaction into the starting point of a guided preparation workflow.

The project also adopts a strictly minimalist user-interface philosophy. Nielsen Norman Group describes aesthetic and minimalist design as the reduction of interface "noise" so that necessary information is emphasized, while W3C accessibility guidance stresses that digital products should remain understandable, operable, and accessible to broad user groups. These principles directly support a simple, linear workflow for booking, recommendation, and checkout.

1.2 Problem Statement

The major problem addressed in this project is the **fragmented trip-preparation process**. Existing travel systems usually stop at transportation and accommodation booking. They do not help users determine what they need to pack or buy for the trip, even though those decisions are heavily influenced by destination, climate, trip duration, and who is traveling.

This fragmentation creates several practical issues. First, users spend excessive time switching among booking websites, weather services, and shopping platforms. Second, users face increased mental load

because they must independently remember destination-specific and demographic-specific needs such as winter clothing, culturally appropriate outfits, children's travel supplies, or pet carriers. Third, users often complete multiple unrelated transactions, which increases friction and reduces convenience.

The problem is therefore not only about missing features; it is about a broken workflow. Travelers do not simply want to “book” and then separately “shop.” They need a unified system that connects both actions logically and efficiently.

1.3 Aim of the Project

The aim of this project is to design and develop a hybrid web platform that integrates travel booking and trip-specific product recommendation into a single, seamless user journey, culminating in a unified checkout interface supported by split-payment routing for travel and retail transactions.

1.4 Objectives of the Study

1. **To develop a web-based booking workflow** that allows users to enter destination, travel dates, party details, and trip purpose, then search and select flight and hotel options through backend API integrations. Amadeus provides APIs for flight and hotel search and booking, making it suitable for the travel-booking component of the system.
2. **To build a rule-based recommendation engine** that generates relevant shopping suggestions using destination, weather conditions, travel month, trip purpose, and traveler demographics as inputs. Open-Meteo provides forecast, historical, and climate-related weather data that can support these recommendation decisions.
3. **To implement a single-interface checkout experience using split payment routing.** In this model, the user experiences one continuous checkout screen, but payment processing is separated according to item type. Travel-booking payments are securely passed through the travel-booking flow required by the airline or hotel provider, while the retail portion of the order is processed on the platform backend through Stripe. This design is more technically realistic than treating travel and retail payments as one captured merchant-of-record payment, because travel APIs and hotel-booking flows commonly require provider-specific payment handling, and Stripe's PaymentIntents model is intended to manage the lifecycle of platform-side payments for custom

checkout flows. Hotel booking through Amadeus, for example, requires credit card payment handling within the booking process, while Stripe PaymentIntents tracks the status of a payment from creation through confirmation.

4. **To design a loyalty and discount mechanism** that rewards frequent users through tier-based reductions, especially on shopping items and, where commercially feasible, on selected service fees or platform-controlled travel commissions rather than directly altering supplier-controlled base fares.

1.5 Motivation for the Study

This project is motivated by three major factors: efficiency, reassurance, and user-centered digital commerce.

First, the system saves time. Instead of repeating search and purchase steps across several websites, the user moves through one connected journey. Second, the system reduces travel anxiety by transforming abstract trip details into concrete preparation guidance. A booked winter trip, for instance, can immediately trigger suggestions for thermal wear, gloves, or waterproof shoes; a family trip can trigger child essentials; and pet-inclusive travel can trigger carrier and feeding accessories. Third, the system promotes more confident travel decision-making, which may encourage users to consider a broader range of destinations because preparation becomes clearer and less stressful.

The project is also motivated by current technical feasibility. Node.js is an open-source, cross-platform JavaScript runtime suited to building servers and web applications, while Express provides a minimal and flexible web framework for routing and middleware. MySQL supports strongly structured relational storage as well as native JSON fields for semi-structured provider payloads. Together, these technologies enable a full JavaScript implementation from server to browser.

1.6 Scope of the Study

The scope of the project covers the design and implementation of a web application with the following capabilities:

- user input of trip destination, dates, party details, and trip purpose;
- flight and hotel search through external provider integration;
- creation of trip sessions and locked travel selections;
- generation of targeted product recommendations based on trip context;
- cart creation containing platform-retail items linked to the booked trip;
- presentation of a **single, seamless checkout interface** in which travel and retail charges are orchestrated together at the user-interface level, but processed through **split payment routing** behind the scenes;
- application of loyalty discounts and checkout pricing to eligible platform-controlled items.

Within this scope, the project does **not** treat all travel and retail items as one internally settled payment ledger. Instead, the application coordinates two connected payment paths. Travel-booking charges are passed to the airline or hotel booking flow as required by the travel provider, while retail-item charges are processed through the platform backend using Stripe. This means the system offers a unified customer experience at the interface layer, while remaining technically consistent with the payment requirements of travel APIs and provider-controlled bookings. Amadeus documentation confirms direct hotel booking support and credit-card payment handling within the hotel booking process, while Stripe documents PaymentIntents as the appropriate mechanism for handling custom platform-side payment flows.

The study is limited to a **web-based platform** built with Node.js, Vanilla JavaScript, HTML, CSS, and MySQL. It does not include native mobile applications, advanced machine-learning recommendation models, or ownership of airline or hotel payment settlement infrastructure. It also does not assume a large, self-warehoused retail operation in its initial version.

1.7 Beneficiaries of the Study

The primary beneficiaries are travelers who want a faster and less stressful planning experience. These include:

- solo travelers who need quick trip preparation support;
- couples who want convenient booking and shopping in one system;
- families traveling with children who need child-specific essentials;
- travelers with pets who require specialized travel supplies;
- first-time travelers who may not know what to prepare;
- and business operators who benefit from combining travel intent with retail conversion.

Because the interface is deliberately minimalist and based on standard web technologies, it is intended to remain accessible to users with different levels of digital literacy. W3C's accessibility guidance supports this emphasis on usable, understandable web content.

1.8 Distinctiveness of the Project

The key distinctiveness of the proposed system is its seamless transition from travel booking to algorithmic packing assistance within a single, minimalist purchase journey. Unlike conventional travel platforms that stop at booking, or e-commerce platforms that lack trip awareness, the proposed system uses booking context as the trigger for tailored retail guidance and presents travel and retail settlement through one integrated interface supported by split-payment routing.

The uniqueness of the platform lies in five combined characteristics:

1. destination-aware recommendations;
2. weather-aware and season-aware filtering;
3. demographic-aware suggestions for men, women, children, and pets;

4. one coordinated cart and checkout experience spanning both travel and retail selections;
5. loyalty discounts that reward repeat platform use.

This combination gives the platform a clear practical identity: it is not merely a travel-booking site and not merely an online store. Rather, it is a preparation-oriented travel commerce system that connects booking intent directly with contextual purchasing support.

1.9 Limitations of the Study

The project has several practical limitations.

First, it depends on third-party services for travel and weather information. If these APIs return incomplete, delayed, or inconsistent responses, the system's outputs may be affected. Amadeus provides travel APIs and Open-Meteo provides weather datasets, but the application still depends on the reliability, availability, and response structure of those external services.

Second, the initial recommendation engine is rule-based rather than machine-learned. This improves transparency and ease of implementation but may not capture every personal preference, brand preference, or exceptional travel case.

Third, the early-stage product inventory is likely to be limited to a curated travel-oriented catalog rather than a full retail marketplace. This means some niche, luxury, or highly localized products may not be available in the first version.

Fourth, a minimalist interface improves clarity, but it may reduce the number of advanced customization, comparison, or itinerary-management features included in the initial release.

Fifth, the e-commerce fulfillment model is intentionally limited in physical operational scope. To avoid the complexity and capital cost of warehouse management, the platform is expected to rely on a **dropshipping model, supplier API feeds, or affiliate-style product sourcing**, rather than holding and dispatching stock from its own warehouse. As a result, product availability, pricing freshness, shipping reliability, and fulfillment speed may depend on external suppliers. This also means some recommended items may become unavailable, delayed, or geographically restricted after recommendation generation, especially when supplier catalogs change faster than the platform cache updates.

1.10 Summary of Chapter One

This chapter established the context, problem, purpose, scope, and uniqueness of the project. The study responds to the fragmented nature of trip preparation by proposing a minimalist web application that integrates travel booking, destination-aware shopping recommendations, and a unified checkout interface supported by split-payment routing. The following chapter reviews related studies and systems relevant to the proposed platform



CHAPTER TWO

RELATED WORKS / LITERATURE REVIEW

2.1 Introduction

This chapter reviews the studies and practical systems most closely related to the proposed PackSwift platform. The project combines three problem domains that have often been studied separately: travel recommendation, context-aware tourism support, and trip-related packing or product assistance. Existing literature shows strong development in destination recommendation, travel-package personalization, itinerary support, and context-aware tourism systems. Commercial tools also show that users value integrated travel planning and contextual packing guidance. However, these capabilities are usually delivered as separate systems rather than as one unified workflow. The proposed project is therefore positioned as a hybrid platform that connects travel booking, trip-aware product recommendation, and coordinated split-payment checkout within one minimalist web application. (Borràs et al., 2014; Felfernig et al., 2006).

2.2 Foundational Studies in Travel Recommendation

An important early contribution is the study by Felfernig et al. (2006), which surveyed recommendation technologies in travel and tourism and explained the use of collaborative, content-based, and knowledge-based approaches in travel decision support. The study is relevant because it established that travel recommendation differs from ordinary retail recommendation: users usually face multiple constraints such as timing, location, budget, and service composition. This insight is directly relevant to PackSwift, where the recommendation process depends on destination, weather, trip purpose, traveler composition, and delivery feasibility rather than on generic shopping history alone. (Felfernig et al., 2006).

A broader and more influential review was later provided by Borràs et al. (2014), who surveyed intelligent tourism recommender systems and showed how artificial intelligence techniques had been increasingly applied to tourism personalization. Their work is significant because it demonstrated that tourism recommenders are most useful when they take context into account rather than presenting static, one-size-fits-all results. This supports the methodological position of PackSwift, which treats trip details as the core source of recommendation logic. At the same time, the reviewed systems mainly focused on attractions, points of interest, and tourism content, rather than on the recommendation of physical goods needed for the trip. (Borràs et al., 2014).

2.3 Personalized Travel Package Recommendation

Liu et al. (2011) proposed a personalized travel package recommendation model that treated travel packages themselves as recommendation items. Their work is especially relevant because it showed that recommendation quality improves when travel data is modeled through destination, season, and package characteristics rather than through simple user-item similarity alone. This idea is highly compatible with the PackSwift concept, where the booked trip becomes the organizing context for all later recommendations. However, Liu et al. focused on recommending travel packages before purchase, while PackSwift extends the logic into the post-booking stage by recommending trip-specific products after flights and hotels have been selected. (Liu et al., 2011).

2.4 Context-Aware Tourism Recommendation

A highly relevant context-aware study is STS by Braunhofer et al. (2013), a weather-aware mobile recommender system for tourism. The authors showed that weather can be treated as an explicit contextual variable in recommendation and that doing so improves the usefulness of tourism suggestions. This is directly related to PackSwift because weather is one of the main signals used in the proposed recommendation engine. The difference is that STS recommended tourism experiences, while PackSwift maps weather and destination context to purchasable goods such as jackets, umbrellas, child winter wear, or waterproof accessories. (Braunhofer et al., 2013).

Namahoot et al. (2015) developed CAT-TOURS, a context-aware Thai tourism recommender system that combined temporal ontology with Naïve Bayes classification. Their study is important because it demonstrated that tourism recommendation becomes more accurate when time and structured contextual knowledge are integrated into the logic. This is relevant to PackSwift because the proposed product recommender also relies on structured contextual inputs, including trip month, departure timing, and trip purpose. However, CAT-TOURS was designed to recommend tourism-related content and websites, not physical products connected to an actual booking workflow. (Namahoot et al., 2015).

A more recent example is the CAFOB system proposed by Abbasi-Moud et al. (2022). Their system combined fuzzy logic and ontology-based reasoning in order to improve context-aware tourism recommendation. The significance of this study lies in its confirmation that contextual richness improves recommendation quality. PackSwift adopts a simpler first-version rule engine instead of fuzzy ontology, but it follows the same underlying principle: recommendation should be shaped by contextual variables rather than by generic catalog browsing alone. (Abbasi-Moud et al., 2022).

Flórez et al. (2025) represent a newer direction in tourism recommendation research through a hybrid method that combines deep learning with ontology-based knowledge. Their work is relevant because it demonstrates how advanced tourism recommenders are moving toward richer, hybrid intelligence that combines learned patterns with structured knowledge. Although PackSwift does not begin with deep learning, this study suggests a credible future pathway for extending the platform beyond rule-based recommendation while retaining explainability through structured trip variables. (Flórez et al., 2025).

2.5 Location-Based and Group Travel Recommendation

Ravi et al. (2016) proposed a collaborative location-based travel recommendation system aimed at improving group travel support through enhanced rating prediction. This study is relevant because it reinforced the importance of location as a central recommendation anchor and also showed that travel decisions are often social rather than purely individual. PackSwift similarly treats destination as more than a search input; destination drives travel offers, climate retrieval, cultural context, and product filtering. While Ravi et al. focused on recommending travel choices

for groups, PackSwift extends this logic to the recommendation of trip-related goods for groups that may include adults, children, and pets. (Ravi et al., 2016).

2.6 Packing and Baggage Recommendation Research

The study closest to the packing-assistant component of PackSwift is the baggage-item recommendation model proposed by Ravi and Negi (2025). Their work used FastText and association-rule mining to generate personalized baggage item recommendations for air travelers. This is especially important for the present project because it directly addresses the practical preparation problem that many travel platforms ignore. The PackSwift concept aligns strongly with this study, but it extends the baggage recommendation idea into a wider system that begins with travel booking, continues through contextual product selection, and ends in a coordinated checkout flow. In this sense, Ravi and Negi's work validates the core problem statement behind PackSwift while leaving room for a broader commerce-oriented implementation. (Ravi & Negi, 2025).

2.7 Related Commercial Systems

In the commercial environment, Expedia demonstrates the popularity of integrated travel-service purchase flows by allowing users to combine flights, hotels, and bundled travel offers. Expedia therefore confirms that users accept consolidated booking workflows. However, its integration remains centered on travel services rather than on post-booking, context-aware retail support. This makes Expedia similar in booking convenience but different in functional scope from PackSwift. (Expedia, n.d.).

Google Travel is another useful comparison because it integrates flight discovery, hotel search, and trip planning functions into one digital environment. Its strength lies in simplifying travel planning and search aggregation. However, it does not convert confirmed trip context into a product recommendation interface tied to physical shopping and retail settlement. This means that Google Travel resembles PackSwift on the discovery side, but not on the packing-support and commerce side. (Google, n.d.).

PackPoint is one of the closest commercial analogues to the recommendation component of the proposed project. It generates packing suggestions based on destination, trip duration, and

weather, which strongly resembles the contextual logic of PackSwift's recommendation engine. Nevertheless, PackPoint is a packing-list tool rather than a booking-and-commerce platform. It does not integrate flight and hotel reservation, trip-linked product purchasing, or coordinated split-payment checkout. Its relevance lies in proving that contextual packing support is useful in practice. (PackPoint, n.d.).

2.8 Research Gap and Positioning of the Proposed System

The reviewed literature and systems show that the proposed platform is grounded in well-established research areas. Tourism studies have shown the value of contextual recommendation, season-aware modeling, weather-aware decision support, package recommendation, and location-based personalization. Commercial platforms have shown that users respond well to integrated travel booking and contextual assistance. Yet these features are usually distributed across separate products. Tourism recommenders mainly support destinations, attractions, or itineraries; packing tools mainly support lists; and travel platforms mainly support reservations. The gap is the absence of a unified system that connects trip booking, trip-context product recommendation, shipping-feasibility filtering, and coordinated split-payment checkout in one minimalist workflow. It is this gap that PackSwift is designed to address. (Borràs et al., 2014; Braunhofer et al., 2013; Liu et al., 2011; Ravi & Negi, 2025).

2.9 Summary of Chapter Two

This chapter reviewed the most relevant studies and real-world systems related to travel recommendation, context-aware tourism support, travel-package personalization, packing assistance, and integrated booking environments. The review showed that substantial progress has already been made in contextual tourism recommendation and practical packing support. However, these systems typically address only one segment of the user journey. The proposed PackSwift platform is positioned as a hybrid extension of this literature, combining the strengths of booking systems, contextual recommendation engines, and packing-assistance tools within one operational travel-commerce workflow. (Abbasi-Moud et al., 2022; Flórez et al., 2025; Ravi & Negi, 2025).

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter explains the technical method used to design and implement the hybrid travel and e-commerce platform. The methodology is intentionally practical and backend-first so that a beginner can understand the order in which the system should be built. The chapter covers architecture, tools, recommendation logic, database design, required datasets, and phased development planning.

3.2 Development Approach

The project follows a **backend-first methodology**. This means database design, routing, business rules, provider integration, and payment flow are implemented before visual interface work begins.

The reason for this approach is straightforward: the application depends on structured relationships among users, trips, products, bookings, carts, orders, and payments. It also depends on outside services for travel and weather data. If the backend is not clearly designed first, the frontend will have no stable foundation.

The backend-first approach therefore proceeds in this order:

1. define the database schema;
2. create the Node.js and Express project structure;
3. build core REST endpoints;

4. connect travel and weather APIs;
5. implement recommendation logic;
6. implement cart, pricing, and checkout logic;
7. then attach the HTML, CSS, and Vanilla JavaScript frontend.

3.3 System Architecture and Design

3.3.1 Architectural Layers

The application is organized into five layers:

1. Presentation Layer

This layer consists of HTML pages, CSS styling, and Vanilla JavaScript. It collects user input, displays booking results, renders recommendations, and handles checkout interaction.

2. Routing Layer

This layer is built with Express. Express is officially described as a minimal and flexible Node.js web application framework, and its middleware-based model is well suited for route handling, validation, authentication checks, and webhook processing.

3. Service Layer

This contains the core business logic: travel search handling, weather retrieval, recommendation scoring, pricing, loyalty calculations, and checkout orchestration.

4. Data Layer

This is the MySQL database, which stores users, destinations, trips, products, bookings, carts, orders, payments, and recommendation logs.

5. Integration Layer

This connects the application to external data providers:

- Amadeus for flights and hotels,
- Open-Meteo for weather and climate context,
- and a payment provider for checkout processing.

Amadeus officially supports flight and hotel APIs, while Open-Meteo provides forecast and historical weather endpoints.

3.3.2 End-to-End Data Flow

The system's data flow is linear and closely aligned with the intended user journey.

Step 1: Trip input

The user enters destination, start date, end date, and party details on a booking form.

Step 2: Frontend request

Vanilla JavaScript sends the form data to the backend using the browser Fetch API. MDN defines Fetch as the standard interface for fetching resources across the network and handling responses.

Step 3: Trip session creation

The backend validates the request and inserts a record into the trip_sessions table. At this point, the trip exists in draft form.

Step 4: Travel search

The backend calls flight and hotel provider endpoints. Returned offers are normalized into internal objects and saved as travel_quotes.

Step 5: Travel selection and lock-in

When the user chooses a flight and hotel, those selections are saved as provisional or confirmed bookings. Matching travel items are also added to the cart.

Step 6: Weather and destination context retrieval

The backend retrieves seasonal or forecast weather for the destination and date range. This data is stored in a weather snapshot associated with the trip.

Step 7: Recommendation generation

The recommendation engine evaluates destination context, weather signals, duration, and traveler demographics. It identifies appropriate product records and ranks them.

Step 8: Shopping interaction

The frontend shows a filtered, narrow product list rather than a general marketplace. This supports minimalist design and reduces unnecessary interface noise.

Step 9: Unified cart and pricing

Selected products and travel items are combined into a single cart. Pricing logic computes subtotals, discounts, and totals.

Step 10: Checkout and payment

The backend creates an order and a payment record, then initiates a payment session. On successful payment confirmation, the order status is updated and downstream records are finalized.

3.4 Technology Stack and Tools

3.4.1 Node.js for Backend Execution

Node.js is used as the runtime environment for backend code. The official Node.js site defines it as a free, open-source, cross-platform JavaScript runtime environment used to create servers, web applications, command-line tools, and scripts. This makes it suitable for a project that aims to remain entirely JavaScript-based.

3.4.2 Express for Routing and Middleware

Express provides the project's HTTP server structure. Its middleware model allows the application to process requests in stages, such as JSON parsing, validation, authentication checks, business-logic delegation, and error handling. The official documentation states that an Express application is essentially a series of middleware function calls.

3.4.3 Vanilla JavaScript, HTML, and CSS for Frontend Delivery

The frontend is intentionally lightweight. HTML provides page structure and semantic forms, CSS provides minimalist layout and readability, and Vanilla JavaScript handles dynamic interactions such as API calls, rendering travel results, showing recommendations, and managing checkout states.

This choice supports clarity, easier learning, and reduced complexity. It also aligns with standard web practices and accessibility expectations.

3.4.4 MySQL for Relational Data Storage

MySQL is used because the application is relationship-heavy. One user may create many trips, one trip may contain many travel quotes and bookings, one cart may include both travel and shopping items, and one order may have several associated order items. MySQL's native JSON data type also allows efficient storage of semi-structured values such as API responses and recommendation snapshots.

3.5 Algorithms and Logic of the Recommendation Engine

3.5.1 Purpose of the Recommendation Engine

The recommendation engine exists to convert trip context into relevant products. It does not behave like a general online store search engine. Instead, it acts as a rules-based packing assistant that uses trip facts to produce a narrow and relevant product set.

3.5.2 Input Variables

The engine uses the following variables:

- destination country and city;
- travel start and end dates;
- travel month;
- **trip purpose** such as Business, Leisure, or Adventure;
- trip duration in days;
- **days until departure**;
- average temperature;
- minimum and maximum temperature;
- precipitation probability or rainy-day estimate;
- snow indicators where relevant;

- destination culture tags;
- number of adult male travelers;
- number of adult female travelers;
- number of children;
- number of pets;
- available product stock quantity;
- **estimated product shipping time** in days;
- product category and tags.

These variables enable the engine to produce recommendations that are not only destination-aware and weather-aware, but also purpose-aware and time-feasible. Weather inputs can be obtained from forecast or historical sources depending on how close the trip is to the current date, while shipping feasibility is derived from supplier-side product metadata stored in the product catalog. Open-Meteo provides the weather signals needed for the environmental part of this decision model.

3.5.3 Product Metadata Required

Each product must contain structured metadata so that trip conditions can be translated into purchasable items. At minimum, each product record should include:

- product ID;
- product name;
- category ID;
- unit price;
- stock quantity;
- active status;
- relevance tags;
- optional attributes in JSON.

Typical product tags include:

- cold_weather
- hot_weather
- rain
- snow

- beach
- modest_clothing
- child_essential
- pet_travel
- electronics
- luggage_accessory

3.5.4 Rule Construction

The first version of the engine is rule-based. This is appropriate because it is easy to explain, test, and audit.

A practical rules sequence is:

Rule A: Temperature bucket

- if average temperature is below 8°C, assign cold_weather
- if average temperature is between 8°C and 21°C, assign mild_weather
- if average temperature is 22°C or above, assign warm_weather

Rule B: Rain relevance

- if precipitation probability is high or rainy days exceed a threshold, assign rain

Rule C: Snow relevance

- if snowfall is present or temperature is near freezing with precipitation, assign snow

Rule D: Cultural relevance

- if destination metadata contains conservative_sites, assign modest_clothing
- if destination metadata contains beach, assign beach and sun_protection

Rule E: Child relevance

- if child count is greater than zero, assign child_essential

Rule F: Pet relevance

- if pet count is greater than zero, assign pet_travel

Rule G: Duration relevance

- if trip duration exceeds seven days, increase the weight for organizers, refillables, repeat-wear basics, or laundry accessories

Rule H: Trip-purpose relevance

- if trip purpose is Business, assign tags such as formal_wear, tech_organizer, laptop_accessory, and business_travel
- if trip purpose is Leisure, assign tags such as casual_wear, comfort_item, and destination-specific recreation tags

- if trip purpose is Adventure, assign tags such as outdoor_gear, durable_footwear, waterproof_item, and activity_accessory

This additional purpose rule ensures that the system does not rely only on climate and traveler composition. Two travelers going to the same city in the same month may still require very different products if one is attending meetings and the other is taking part in outdoor activities.

3.5.5 Product Matching Logic

After the engine generates a set of trip tags, it queries MySQL for all active, in-stock products whose tags match one or more of those trip tags.

A revised matching sequence is:

1. generate trip tags from weather, culture, traveler composition, and trip purpose;
2. fetch all products where active = true and supplier availability indicates that the item can still be ordered;
3. exclude all products where stock_qty <= 0;
4. apply a **shipping-feasibility filter** by excluding every physical product whose estimated_shipping_days is greater than the number of days_until_departure;
5. join the remaining products with product-tag mapping records;
6. keep products whose tags intersect with the generated trip-tag set;
7. assign a relevance score to each candidate;
8. sort descending by score;

9. return the top-ranked products.

The shipping-feasibility filter is strict because recommendation relevance is not only about need, but also about deliverability before departure. A winter jacket that arrives after the traveler has already left is not practically relevant, even if its weather score is high. For that reason, shipping feasibility must be treated as a hard exclusion rule before scoring begins, not as a minor ranking preference.

3.5.6 Scoring Formula

A practical scoring formula is:

$$\text{final_score} = (\text{weather_score} \times 0.40) + (\text{culture_score} \times 0.20) + (\text{traveler_score} \times 0.20) + (\text{duration_score} \times 0.10) + (\text{inventory_score} \times 0.10)$$

Where:

- weather_score measures fit to the climate bucket,
- culture_score measures fit to destination norms,
- traveler_score measures relevance to adults, children, or pets,
- duration_score measures usefulness for longer stays,
- inventory_score boosts available and stable-stock items.

3.5.7 Example

Consider this trip:

- destination: Toronto;
- month: January;
- weather: cold with snow;
- travelers: two adults, one child, one pet.

The engine may produce these trip tags:

- cold_weather
- snow
- child_essential
- pet_travel

These tags would then match products such as:

- Product 104: thermal jacket
- Product 118: insulated gloves
- Product 145: child winter boots
- Product 208: pet carrier

This logic ensures that recommendations are explainable rather than arbitrary.

3.6 Database Schema and Storage Design

3.6.1 Core Tables

The database should contain the following major tables:

- users
- loyalty_tiers
- traveler_profiles
- destinations
- trip_sessions
- travel_quotes
- bookings
- product_categories
- products
- product_tags
- product_tag_map
- product_rules

- carts
- cart_items
- orders
- order_items
- payments

3.6.2 Relationship Structure

The main relationships are:

- one user to many trip sessions;
- one destination to many trip sessions;
- one trip session to many travel quotes;
- one trip session to many bookings;
- one trip session to one active cart;
- one cart to many cart items;
- one product category to many products;
- many products to many tags through a mapping table;
- one cart to one order;
- one order to many order items;
- one order to one or more payments.

This design preserves traceability from trip planning through final purchase.

In the revised design, the trip_sessions table should explicitly store trip purpose so that the recommendation engine can apply purpose-based rules during recommendation generation. The products table should explicitly store estimated shipping days so that the engine can remove products that cannot realistically arrive before departure. This preserves the relational traceability

of recommendation decisions while keeping the matching logic practical and operationally realistic.

3.6.3 Why Relational Storage Is Appropriate

A relational design is appropriate because the application involves strict transactional relationships. Prices, stock quantities, booking references, and payment statuses must remain consistent and queryable. MySQL's structured tables are therefore used for all primary business entities, while JSON columns are reserved for flexible values such as raw travel-provider payloads, weather snapshots, or rule snapshots. MySQL's JSON data type automatically validates JSON documents and supports JSON-oriented functions, which makes it suitable for this hybrid storage strategy.

3.7 Datasets and External Resources

3.7.1 Travel Data

The system requires external travel data for:

- airport codes,
- flight offers,
- hotel offers,
- booking references,
- destination content.

Amadeus provides official self-service APIs for flights and hotels, making it an appropriate travel data source for this project.

3.7.2 Weather and Seasonal Data

The system requires weather context including:

- short-term forecasts,
- historical seasonal patterns,
- temperature ranges,
- rainfall likelihood,
- and snowfall indicators.

Open-Meteo provides forecast, historical weather, and large-scale climate data suitable for this purpose.

3.7.3 Product Catalog Data

The platform also requires an internal product dataset. At the prototype stage, this can be seeded manually using a dummy catalog. The catalog should include:

- product ID,
- name,
- category,
- price,
- currency,

- stock quantity,
- tags,
- image path,
- description,
- optional attributes.

Initial categories should include:

- luggage accessories,
- weather clothing,
- footwear,
- child travel items,
- pet travel items,
- toiletries and organizers,
- adapters and travel basics.

3.7.4 Loyalty Data

The system should be seeded with tier records such as:

- Bronze,
- Silver,
- Gold,
- Platinum.

Each tier should define:

- minimum completed orders,
- minimum lifetime spend,
- shopping discount percentage,
- optional travel discount percentage.

3.8 Development Planning

3.8.1 Phase One: Backend Foundation — Database Schema and Node.js Server Setup

This phase establishes the technical foundation of the entire platform. It defines the MySQL relational schema, initializes the Node.js and Express server, and prepares stable API structures for all later integrations.

At this stage, the developer should:

- initialize the Node.js project and create package.json;
- configure the project directory structure, including config, db, routes, controllers, services, repositories, middleware, and public;
- install required backend dependencies for Express, MySQL connectivity, environment variable management, JSON parsing, and error handling;
- create the main application entry file;
- configure Express middleware for JSON request parsing, URL-encoded form parsing, request logging, and centralized error handling;
- establish MySQL database connection settings;
- create the database schema with the core tables: users, loyalty_tiers, traveler_profiles, destinations, trip_sessions, travel_quotes, bookings, product_categories, products, product_tags, product_tag_map, carts, cart_items, orders, order_items, and payments;
- define foreign-key relationships between users, trips, bookings, products, carts, orders, and payments;
- add fields needed for recommendation and delivery logic, including trip_purpose in trip_sessions and estimated_shipping_days in products;
- seed initial data for loyalty tiers, destinations, product categories, tags, and sample products;
- create repository modules for database access abstraction;
- create initial Express routes for authentication and trip-session creation;
- ensure all foundational routes return structured JSON responses.

Deliverable:

A working backend foundation consisting of an initialized MySQL database, a running Node.js and Express server, and stable base API endpoints connected to the database.

Technical Justification:

This phase must occur first because every later service depends on a stable, queryable relational data model and a functioning backend runtime environment.

System Architecture Structure for Phase One

Component	Function
Client Request Source	Provides future frontend or API input to the backend
Express Routes	Accept incoming requests and map them to application logic
Controllers	Validate requests and organize input before service processing
Services	Execute business logic and coordinate backend operations
Repositories	Perform SQL queries and database updates
MySQL Database	Stores persistent application data in structured tables

Process Flow for Phase One

1. The backend receives an incoming request.
2. Express routes direct the request to the correct controller.
3. The controller validates and organizes the input.
4. The service layer applies business rules where necessary.

5. The repository layer communicates with MySQL.
6. The database stores or retrieves the requested data.
7. A structured JSON response is returned.

3.8.2 Phase Two: Backend Logic — Travel APIs and Recommendation Engine

This phase enables the platform to search flights and hotels, capture trip context, retrieve weather information, and generate algorithmic packing suggestions from rule-based logic. At this stage, the platform moves beyond static data storage and begins to perform its core functional purpose: transforming trip information into travel options and contextual retail recommendations.

At this stage, the developer should:

- configure Amadeus API credentials in the backend environment;
- create a dedicated travel service module for provider communication;
- implement token retrieval and refresh handling for Amadeus requests;
- create destination lookup logic to normalize city and airport data;
- implement flight-search service methods using origin, destination, departure date, return date, and traveler details;
- implement hotel-search service methods using destination, check-in and check-out dates, and guest count;
- normalize Amadeus responses into platform-specific JavaScript objects;
- store raw provider payloads in `travel_quotes` for traceability and debugging;
- create Express routes for flight search, hotel search, and travel lock-in;
- when travel is selected, create booking records and update trip status;
- configure Open-Meteo integration for contextual weather retrieval;
- build a weather service that retrieves forecast data, seasonal averages, temperature values, precipitation values, and snow indicators;
- define recommendation inputs including destination, trip month, trip purpose, days until departure, weather conditions, adult count, child count, pet count, stock quantity, and estimated shipping days;
- write rule functions for cold weather, rain, snow, children, pets, duration, and trip purpose;

- implement shipping-feasibility exclusion logic that removes any product whose estimated shipping time exceeds the remaining days before departure;
- create product matching and scoring logic;
- create routes for recommendation generation and recommendation retrieval;
- persist recommendation outputs for traceability, evaluation, and debugging.

Deliverable:

A functioning backend capable of searching flights and hotels, locking selected travel options, retrieving trip weather context, and generating relevant, time-feasible product recommendations.

Technical Justification:

Travel and recommendation logic must be completed before cart and payment work begins, because product suggestions and later pricing depend on confirmed trip context and valid product eligibility.

System Flow Structure for Phase Two

Step	System Activity
1	A trip session is created and stored in the database
2	The backend sends travel search requests to Amadeus
3	Flight and hotel responses are normalized and stored as travel quotes
4	The user selects travel options and the system creates booking records
5	The backend retrieves weather and seasonal context from Open-Meteo

6	The recommendation engine generates context tags from travel, weather, and traveler data
7	The system filters products using stock and shipping-feasibility rules
8	The engine scores and ranks matched products
9	Recommendation results are returned to the frontend and stored for traceability



Recommendation Logic Inputs for Phase Two

Input Variable	Purpose
Destination	Determines geographic and cultural relevance
Travel Month	Determines seasonal conditions
Trip Purpose	Distinguishes business, leisure, or adventure needs
Days Until Departure	Filters products by delivery feasibility
Adult / Child / Pet Count	Determines demographic-specific product needs
Weather Data	Determines climate suitability
Product Tags	Enables rule-based matching
Estimated Shipping Days	Excludes products that cannot arrive in time

3.8.3 Phase Three: Backend Transactions — Cart, Pricing, and Split-Payment Routing

This phase implements transactional logic for combining travel selections and retail items within a single interface while routing payment according to item type. At this stage, the system transitions from recommendation to commercial execution, integrating carts, loyalty discounts, order creation, and split-payment handling.

At this stage, the developer should:

- create a cart service that initializes a cart after travel selection is locked;
- insert locked flight and hotel items into cart records;
- create cart routes for viewing, adding, updating, and removing items;
- enable recommended retail products to be added into the cart;
- create pricing logic to calculate travel subtotal, retail subtotal, shipping cost, loyalty discount, and final totals;
- tag each cart item by payment responsibility, distinguishing provider-routed travel items from platform-routed retail items;
- configure Stripe credentials and create Stripe service methods for payment initialization, PaymentIntent creation, payment confirmation, and webhook handling;
- implement checkout routes for order preparation, payment-intent generation, and order confirmation;
- create logic that routes flight and hotel charges to the required provider booking flow while routing retail charges through Stripe on the platform backend;
- create orders and payments records and store split-payment breakdowns;
- implement failure handling for provider-payment failure, Stripe-payment failure, and partial success states;
- update loyalty logic based on completed orders and shopping activity.

Deliverable:

A complete transactional backend supporting trip-linked cart management, loyalty-aware pricing, and split-payment routing with provider-directed travel settlement and Stripe-based retail payment processing.

Technical Justification:

Cart, pricing, and payment logic must be stable before frontend integration because the user interface can only present meaningful checkout screens after totals, routing rules, and transaction states are correctly implemented.

Payment Routing Structure for Phase Three

Payment Category	Processing Method	System Responsibility
Flight Booking Charges	Routed through provider booking flow	Coordinated by backend, settled by travel provider
Hotel Booking Charges	Routed through provider booking flow	Coordinated by backend, settled by travel provider
Retail Product Charges	Processed through Stripe	Managed directly by platform backend
Loyalty Discount	Applied only to eligible platform-controlled items	Calculated by pricing service
Shipping Charges	Added to retail subtotal where applicable	Calculated by pricing service

Transactional Flow for Phase Three

1. Travel selections and recommended products are stored in the cart.
2. The pricing engine calculates subtotals and discounts.
3. The backend separates travel items from retail items by payment responsibility.
4. Provider-routed items are prepared for supplier-side payment handling.
5. Platform-routed items are prepared for Stripe processing.
6. Order and payment records are stored in MySQL.
7. Payment success or failure updates the order lifecycle.

3.8.4 Phase Four: Frontend Interface — Trip Input and Travel Selection Screen

This phase creates the first user-facing interface: a minimalist screen where users enter trip details, search travel options, and select flights and hotels. The design follows the project's minimalist philosophy by using a linear structure, low visual density, and clearly prioritized actions.

At this stage, the developer should:

- create the HTML structure for the trip search page;
- build a minimalist top bar with the platform title and a progress indicator;
- create a structured trip input form with fields for origin, destination, departure date, return date, trip purpose, adults, children, and pets;
- style the page using CSS with wide spacing, restrained visual hierarchy, and clear primary buttons;
- write Vanilla JavaScript to capture form input, submit trip creation requests, trigger flight and hotel searches, and render returned travel options dynamically;
- create travel result cards for flights and hotels;
- implement one-click selection of a flight and a hotel;
- add inline validation and clear empty-state feedback;
- preserve minimalist design by avoiding heavy menus, sidebars, or unnecessary filter groups.

Deliverable:

A working trip-input and travel-selection screen connected to the backend travel endpoints.

Technical Justification:

This is the first frontend phase because it depends directly on stable trip and travel-search APIs produced in the earlier backend stages.

Interface Structure for the Trip Input and Travel Selection Screen

Screen Area	Content
Header	Platform name and progress step
Main Title Area	Brief instruction introducing trip planning
Trip Input Form	Origin, destination, dates, trip purpose, adult count, child count, pet count
Search Action Area	Main button for searching flights and hotels
Flight Results Section	List of available flight cards with selection buttons
Hotel Results Section	List of available hotel cards with selection buttons
Navigation Action Area	Continue button leading to recommendations

User Interaction Flow for Phase Four

1. The user enters trip information.
2. The frontend submits the form to create a trip session.
3. The user requests flight and hotel search results.
4. The frontend displays normalized travel options from the backend.
5. The user selects a flight and hotel.
6. The system stores the selections and prepares the next stage.

3.8.5 Phase Five: Frontend Interface — Smart Packing Recommendations Screen

This phase presents the recommendation output as a highly filtered shopping interface tailored to the user's booked trip. The screen is designed to act as a packing assistant rather than a general e-commerce catalog. Its structure emphasizes clarity, limited choice overload, and visible reasons for recommendation.

At this stage, the developer should:

- create the HTML structure for the recommendation page;
- display a trip summary strip containing destination, dates, trip purpose, traveler composition, and weather snapshot;
- use Vanilla JavaScript to request recommendation data from the backend;
- render product cards showing product image placeholder, product name, reason tag, price, shipping-feasibility note, and add-to-cart button;
- group recommendations visually into minimalist categories such as essential items, weather-related items, and family or pet items;
- add a compact cart preview area showing selected item count;
- keep the interface visually light through generous spacing and simple labels;
- disable or hide products that fail shipping-feasibility rules.

Deliverable:

A working recommendation screen that displays trip-specific, rule-generated product suggestions and allows items to be added to the cart.

Technical Justification:

This screen depends on the completed backend recommendation engine and cannot be built meaningfully until recommendation outputs are stable, filtered, and explainable.

Interface Structure for the Smart Packing Recommendations Screen

Screen Area	Content
Header	Platform name and progress step
Trip Summary Strip	Destination, travel dates, trip purpose, traveler composition, and weather snapshot
Recommendation Introduction	Short explanation of why the listed items were selected
Product Grid	Product cards with image, title, reason tag, price, shipping note, and add-to-cart action
Cart Preview	Small area showing the current selected item count
Recommendation Reason Panel	Brief text explaining the rule factors behind the recommendations
Navigation Action Area	Back button and continue-to-checkout button

Recommendation Screen Logic Structure

Recommendation Group	Typical Items
Weather-Based Items	Jackets, gloves, waterproof shoes, umbrellas
Purpose-Based Items	Formal wear, tech organizers, outdoor gear
Child-Related Items	Child shoes, travel kits, child accessories
Pet-Related Items	Pet carriers, portable bowls, pet travel accessories

3.8.6 Phase Six: Frontend Interface — Unified Checkout Screen

This phase creates the final checkout screen, where the user reviews travel items and retail items in one minimalist interface while the system transparently handles split-payment processing. The goal is to preserve a seamless user experience even though payment execution is divided between provider-routed travel settlement and platform-routed retail payment.

At this stage, the developer should:

- create the checkout page HTML layout;
- build sections for trip summary, travel items, retail items, pricing summary, and payment explanation;
- use Vanilla JavaScript to fetch cart and order-preparation data;
- render travel line items separately from retail line items while preserving one-screen continuity;
- clearly indicate that travel booking charges are processed through provider booking flow, while retail charges are processed securely by the platform payment system;
- render loyalty discount lines only where applicable;
- create the payment form area for Stripe retail checkout;
- create action buttons for confirming travel booking, paying the retail portion, and finalizing the order;
- add status messaging for partial failures or retry situations;
- use minimalist CSS to preserve clarity and reduce cognitive load during payment.

Deliverable:

A complete checkout screen that presents one seamless user experience while clearly communicating the internal split-payment structure.

Technical Justification:

This is the final frontend phase because it depends on completed cart logic, pricing calculations, payment routing, and stable backend checkout states.

Interface Structure for the Unified Checkout Screen

Screen Area	Content
Header	Platform name and progress step
Checkout Introduction	Short explanation that the user is reviewing and completing the trip
Trip Summary Panel	Destination, dates, purpose, and traveler summary
Travel Booking Section	Flight and hotel items with provider-payment explanation
Retail Items Section	Selected products, shipping, loyalty discount, and subtotal
Pricing Summary Panel	Travel total, retail total, discount, shipping, and final total
Payment Section	Payment form for platform-routed retail items
Action Area	Confirm travel booking, pay retail portion, and finalize order

Checkout Communication Structure

Checkout Element	Purpose
Travel Booking Note	Explains that airline and hotel charges are routed through travel-provider flows
Retail Payment Note	Explains that physical product payment is handled by Stripe
Pricing Summary	Shows the full cost breakdown in a simple format
Final Action Buttons	Allows the user to complete the final transaction sequence clearly

3.9 Summary of Chapter Three

This chapter presented the methodology for designing and implementing the proposed hybrid travel and e-commerce platform. The method follows a backend-first strategy beginning with relational database design, server architecture, routing, third-party API integration, and rule-based recommendation logic, before progressing to cart orchestration, split-payment routing, and frontend delivery. The selected stack of Node.js, Express, Vanilla JavaScript, HTML, CSS, and MySQL provides a consistent and practical basis for building a full end-to-end prototype. The next stage of the project is implementation, testing, and refinement of the system against the functional and usability objectives established in the earlier chapters.

References

- Abbasi-Moud, Z., Hosseinabadi, S., Kelarestaghi, M., & Eshghi, F. (2022). CAFOB: Context-aware fuzzy-ontology-based tourism recommendation system. *Expert Systems with Applications*, 198, 116877. <https://doi.org/10.1016/j.eswa.2022.116877>
- Amadeus for Developers. (n.d.). *Amadeus for Developers: Connect to Amadeus travel APIs*. Amadeus. Retrieved April 28, 2026, from <https://developers.amadeus.com/>
- Amadeus for Developers. (n.d.). *Flight APIs | Search, booking, airlines and airports*. Amadeus. Retrieved April 28, 2026, from <https://developers.amadeus.com/self-service/category/flights>
- Amadeus for Developers. (n.d.). *Get started with Amadeus APIs*. Amadeus. Retrieved April 28, 2026, from <https://developers.amadeus.com/get-started/get-started-with-amadeus-apis-334>
- Borràs, J., Moreno, A., & Valls, A. (2014). Intelligent tourism recommender systems: A survey. *Expert Systems with Applications*, 41(16), 7370–7389. <https://doi.org/10.1016/j.eswa.2014.06.007>
- Braunhofer, M., Elahi, M., Ge, M., Ricci, F., & Schievenin, T. (2013). STS: Design of weather-aware mobile recommender systems in tourism. In *Proceedings of the First International Workshop on Information Technology and Tourism*.
- Expedia. (n.d.). *Travel: Cheap hotels, air tickets & packages*. Retrieved April 28, 2026, from <https://www.expedia.com/>
- Express.js. (n.d.). *Express - Node.js web application framework*. Retrieved April 28, 2026, from <https://expressjs.com/>
- Express.js. (n.d.). *Routing*. Retrieved April 28, 2026, from <https://expressjs.com/en/guide/routing.html>
- Express.js. (n.d.). *Using middleware*. Retrieved April 28, 2026, from <https://expressjs.com/en/guide/using-middleware.html>
- Felfernig, A., Gordea, S., Jannach, D., Teppan, E., & Zanker, M. (2006). A short survey of recommendation technologies in travel and tourism. *ÖGAI Journal*, 25(2), 17–22.
- Flórez, M., Carrillo, J., & Rodríguez, J. (2025). A context-aware tourism recommender system using a hybrid method combining deep learning and ontology-based knowledge. *Journal of Theoretical and Applied Electronic Commerce Research*, 20(3).
- Google. (n.d.). *Travel with Google*. Retrieved April 28, 2026, from <https://travel.google/>

Liu, Q., Ge, Y., Li, Z., Chen, E., & Xiong, H. (2011). Personalized travel package recommendation. In *2011 IEEE 11th International Conference on Data Mining* (pp. 407–416). IEEE. <https://doi.org/10.1109/ICDM.2011.118>

McKinsey & Company. (2024, May 29). *Now boarding: Faces, places, and trends shaping tourism in 2024*.

McKinsey & Company. (2024, May 29). *The state of tourism and hospitality in 2024*.

Mozilla Developer Network. (n.d.). *Fetch API*. MDN Web Docs. Retrieved April 28, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Mozilla Developer Network. (n.d.). *Using the Fetch API*. MDN Web Docs. Retrieved April 28, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Mozilla Developer Network. (n.d.). *Window: fetch() method*. MDN Web Docs. Retrieved April 28, 2026, from <https://developer.mozilla.org/en-US/docs/Web/API/Window/fetch>

MySQL. (n.d.). *JSON data type*. Oracle. Retrieved April 28, 2026, from <https://dev.mysql.com/doc/refman/en/json.html>

Namahoot, C. S., Brückner, M., & Panawong, N. (2015). Context-aware tourism recommender system using temporal ontology and Naïve Bayes. In H. Unger, P. Meesad, & S. Boonkrong (Eds.), *Recent Advances in Information and Communication Technology 2015* (pp. 183–194). Springer. https://doi.org/10.1007/978-3-319-19024-2_19

Nielsen Norman Group. (2021, January 24). *Aesthetic and minimalist design (Usability Heuristic #8)*.

Nielsen Norman Group. (2024, January 30). *10 usability heuristics for user interface design*.

Node.js. (n.d.). *About Node.js*. Retrieved April 28, 2026, from <https://nodejs.org/en/about>

Node.js. (n.d.). *Introduction to Node.js*. Retrieved April 28, 2026, from <https://nodejs.org/learn/getting-started/introduction-to-nodejs>

Node.js. (n.d.). *Node.js*. Retrieved April 28, 2026, from <https://nodejs.org/en>

Open-Meteo. (n.d.). *Climate API documentation*. Retrieved April 28, 2026, from <https://open-meteo.com/en/docs/climate-api>

Open-Meteo. (n.d.). *Historical weather API documentation*. Retrieved April 28, 2026, from <https://open-meteo.com/en/docs/historical-weather-api>

Open-Meteo. (n.d.). *Open-Meteo: Free open-source weather API*. Retrieved April 28, 2026, from <https://open-meteo.com/>

Open-Meteo. (n.d.). *Weather forecast API documentation*. Retrieved April 28, 2026, from <https://open-meteo.com/en/docs>

PackPoint. (n.d.). *PackPoint travel packing list app*. Retrieved April 28, 2026, from <https://www.packpnt.com/>

Ravi, L., & Vairavasundaram, S. (2016). A collaborative location based travel recommendation system through enhanced rating prediction for the group of users. *Computational Intelligence and Neuroscience*, 2016, 1291358. <https://doi.org/10.1155/2016/1291358>

Ravi, M., & Negi, A. (2025). *A multi-tiered solution for personalized baggage item recommendations using FastText and association rule mining* (arXiv:2501.09359). arXiv. <https://arxiv.org/abs/2501.09359>

Stripe. (n.d.). *Create a PaymentIntent*. Retrieved April 28, 2026, from https://docs.stripe.com/api/payment_intents/create

Stripe. (n.d.). *Payment Intents*. Retrieved April 28, 2026, from https://docs.stripe.com/api/payment_intents

Stripe. (n.d.). *The Payment Intents API*. Retrieved April 28, 2026, from <https://docs.stripe.com/payments/payment-intents>

World Wide Web Consortium. (2025, May 6). *Web Content Accessibility Guidelines (WCAG) 2.1*. Retrieved April 28, 2026, from <https://www.w3.org/TR/WCAG21/>

World Wide Web Consortium, Web Accessibility Initiative. (n.d.). *WCAG 2 overview*. Retrieved April 28, 2026, from <https://www.w3.org/WAI/standards-guidelines/wcag/>